

keyRotationTool

Access key rotation tool (public/key-rotation.html)

What this is

A single-file, dependency-free browser page that lets a broadcaster rotate the AWS access key they use to upload Diamond 2 XML files. It calls the AWS IAM API directly from the browser using hand-rolled SigV4 signing over the Web Crypto API. There is no back end.

Context: TEP's preferred authentication method is cross-account `sts:AssumeRole`, but some broadcasters have no AWS account and cannot use it. For those broadcasters TEP issues a long-lived IAM user access key and grants that user permission to manage its own access keys, so the broadcaster can rotate on their own schedule with no input from TEP. Rotation is contractually required at least every 12 months.

The page serves two audiences:

1. **Integration-Lite uploader users.** The file lives in `public/`, so Vite copies it byte-for-byte into `dist/` and it is served from the uploader's own origin at `/key-rotation.html` (linked, deliberately quietly, from the Settings modal). Because it shares the uploader's origin, it reads the key saved in the uploader's settings (`localStorage`) to prefill step 1, and writes the new key back after verification — so the uploader switches to the new key automatically and a test upload through the real app becomes the “verify before delete” step.
2. **Broadcasters who don't use Integration-Lite** (or who won't trust a hosted page). They visit the same URL and type their key in, or save the file and run it from `file://`, where it finds no `localStorage` and behaves as a fully standalone tool with no persistence at all.

One file, both modes, no separate versions to maintain.

Why it is built this way

Single file, no build step, no framework. So it can be audited as one artefact and, for the cautious, saved and run locally from `file:///`. Vite's `public/` directory copies it verbatim — the deployed file is byte-identical to the file in the repo. **Do not move it into the Svelte build or introduce any dependency.**

Hand-rolled SigV4 rather than `@aws-sdk/client-iam` (even though the uploader already bundles SDK v3). SDK v3 injects `amz-sdk-invocation-id` and `amz-sdk-request-headers`, and it is not confirmed that the IAM endpoint's CORS `Access-Control-Allow-Headers` permits them. Hand-signing keeps the header set to exactly `authorization` and `x-amz-date`, which is confirmed to work (see verification notes below).

No STS. `ListAccessKeys` is called with no `UserName` parameter, so IAM infers the user from the signing credential. This keeps the required permission set to exactly the four actions TEP grants.

Deactivate before delete, with a reactivate escape hatch. Deletion is irreversible. The UI forces deactivate, then a real-world upload test, then delete. Reactivate stays available until deletion — and if the tool wrote the new key into the uploader's settings, reactivating also restores the old key there, so the app and AWS always

agree on which key is in service. Preserve this sequencing.

Write-back ordering. When a stored key is being rotated, the new key is written to localStorage immediately after it is verified against IAM — *before* the old key is deactivated or deleted. A closed tab mid-flow can therefore never leave the uploader holding a dead key.

The embedded logo. The CDN logo is inlined as a base64 data URI so the file stays self-contained (CSP allows `img-src data: only`). It was generated once at authoring time, not at build time, so repo and deployment stay byte-identical. To regenerate after a logo change:

```
convert public/cdn-logo-white.png -resize x64 /tmp/logo64.png
base64 -w0 /tmp/logo64.png # paste into the data URI in the <img> tag
```

CORS verification (done 26 August 2026)

Tested from origin <https://taphub.creativediversitynetwork.com> against <https://iam.amazonaws.com/>:

1. **Simple request, no preflight** — unsigned POST with Content-Type: `application/x-www-form-urlencoded`. Result: 403 (correct, unsigned) with Access-Control-Allow-Origin: * and Date in Access-Control-Expose-Headers (the clock-skew correction relies on this).
2. **Preflight for a signed request** — adding X-Amz-Date and Authorization forces an OPTIONS preflight. Result: 200 with Access-Control-Allow-Headers: `authorization,x-amz-date`, Access-Control-Allow-Methods: `POST`, Access-Control-Max-Age: `172800`.

Caveat: the returned Access-Control-Allow-Headers was identical to the request headers sent, so it may be an echo rather than a fixed allowlist. If you ever need to add a header to the signed request set, test the preflight for it first.

Hard constraints — do not break these

- **No external requests except <https://iam.amazonaws.com>.** The CSP meta tag enforces `default-src 'none'` (plus `img-src data:` for the inlined logo). No fonts, analytics, CDNs, external stylesheets or scripts. The disclosure panel in the UI makes this claim to security reviewers, so it must remain true.
- **localStorage access is limited to exactly two keys** — `tep_aws_access_key_id` and `tep_aws_secret_access_key`, shared with the uploader (`src/stores.js`). Bucket, region and every other `tep_*` key are out of bounds. All access is wrapped in try/catch and every failure mode degrades to the standalone behaviour: manual entry, no persistence. No sessionStorage, cookies, IndexedDB, or query-string parameters. In-memory credentials are wiped on `pagehide`.
- **Never set credentials: "include" on fetch.** Wildcard Access-Control-Allow-Origin is incompatible with credentialed mode; SigV4 authenticates via the Authorization header.
- **The signed Content-Type must match the sent Content-Type byte for byte,** including `; charset=utf-8`. This is the most likely cause of `SignatureDoesNotMatch`.
- **Signing region is us-east-1, service iam,** regardless of the bucket's region. IAM is a global service signed against `us-east-1`.
- **Only these four IAM actions:** `ListAccessKeys`, `CreateAccessKey`, `UpdateAccessKey`, `DeleteAccessKey`. The page must never touch S3 or any other service.

Behaviours worth preserving

- **Clock sync.** One unsigned POST before the first signed call reads the Date response header and computes an offset applied to all signing, so drifted workstation clocks don't produce opaque signature failures.

- **Eventual consistency.** New keys are not immediately usable; verification retries ten times at 1.5s intervals and on failure explicitly tells the user not to delete the old key.
- **Two-key limit** rendered as two literal slots; creation disabled when both are full.
- **Retire controls on the other slot.** When both slots are full and no key has been created in this session, the slot the user is *not* signed in with carries deactivate / reactivate / delete buttons, with a hint that reads the key's status and age (inactive → just delete; newer than the signed-in key → probably an abandoned rotation; older → may belong to another system, so check before retiring). This is how a user recovers after closing the tab midway through a rotation. Delete is only enabled once the key is inactive, matching step 5. The signed-in key's slot never gets these controls, so the user cannot lock themselves out. This path never touches localStorage.
- **Age warning at 330 days** — a month's notice before the 12-month contractual limit.
- **Error translation.** AWS error codes map to actionable guidance. Keep new errors in this style: say what went wrong and what to do about it, never apologise, never be vague.

Not yet done

1. **Never tested against a live AWS credential.** The SigV4 canonicalisation is correct by construction but unproven end to end. Test a full rotation against a disposable IAM user before any broadcaster uses it.
2. No automated tests.
3. No accessibility audit beyond visible focus rings and prefers-reduced-motion.
4. The broadcaster-facing rotation instructions that TEP will reference do not exist yet.